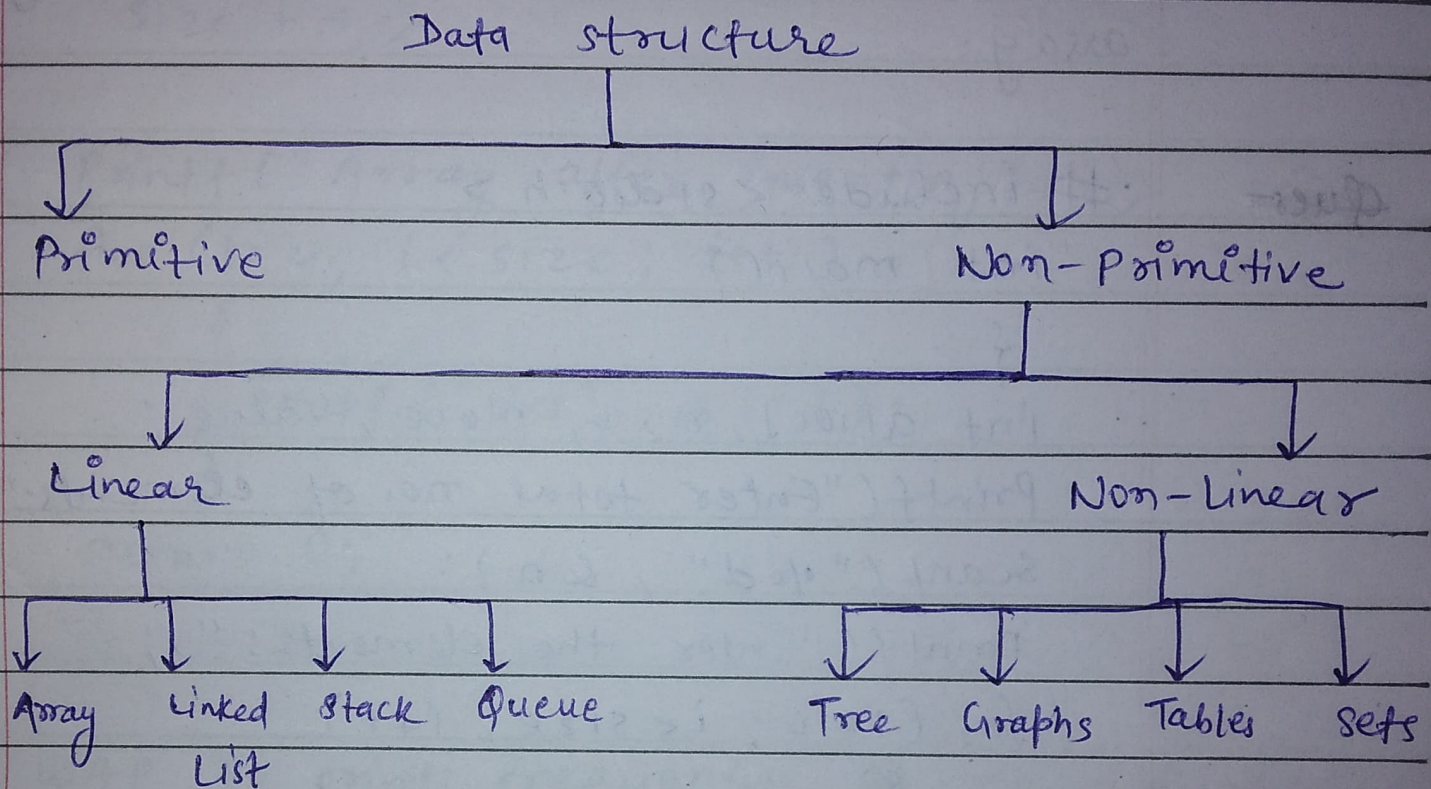


Classification of Data Structure



Ques. What is the difference between Structure, data structure and file structure? Give an example each.

Ques. What is data structure? Classify at different levels.

Ques. write a program to delete a particular number in a given array.

Ques. write a program to insert any particular number at particular position in a given array.

```
#include <stdio.h>
int main()
{
    int a[100], n, i, place, value;
    printf("Enter total no. of elements:");
    scanf("%d", &n);
    printf("Enter the elements:");
    for (i = 0; i < size; i++)
    {
        scanf("%d", &a[i]);
    }
    printf("Position of number:");
    scanf("%d", &place);

    printf("Enter the number:");
    scanf("%d", &value);
    for (i = n; i >= place; i--)
    {
        a[i] = a[i-1];
```

```
    }
    a[place-1] = value;
    size++;

    printf("Array after inserting:");
    for (i = 0; i < size; i++)
    {
        printf("%d", a[i]);
    }
    return 0;
}
```

Ques. WAP to print frequency of each value in a given array.

Analysis of an Algorithm

Algorithm notations are a mathematical model used to describe and classify the efficiency of an algorithm based on its input size (n). It focuses on the growth rate of the algorithm performance ignoring constant factors and lower order term.

There are three main Asymptotic/algorithm notations:-

- (i) Big O notation ($O(n)$)
- (ii) Omega notation ($\Omega(n)$)
- (iii) Theta notation ($\Theta(n)$)

(i) $O(n)$:

- It described as worst case of an algorithm.
- It define upper bound of an algorithm.
- It indicate the maximum time the algorithm will ever require, even in the worst-case scenario.
- If $f(n) \leq c \cdot g(n) \forall n \geq n_0, c > 0, n_0 \geq 1$

$f(n)$: The actual running time function of the algorithm.

$g(n)$: The simplified asymptotic function.

c : A positive constant.

n_0 : A minimum input size.

(ii) $\Omega(n)$

- It is described as the best case of an algorithm.
- It defines the lower bound for the algorithm's running time.
- It indicates the minimum time the algorithm will ever require, regardless of the input.
- If $f(n) \geq c \cdot g(n) \forall n \geq n_0, c > 0, n_0 \geq 1$

(iii) $\Theta(n)$

- It described as the average case of an algorithm.

- It defines the average bond of an algorithm having both lower as well as upper bond.
- It indicates average time required by an algorithm.
- If $c_1.g(n) \leq f(n) \leq c_2.g(n) \forall n \geq n_0$, $c_1 > 0$, $c_2 > 0$ and $n_0 \geq 1$.

Ques. what is asymptotic notations? Discuss the best case, average case and worst case notations of an algorithm.

STACK

Stack is a linear data structure that follows the principle of LIFO (Last In First Out) where we put a new element at the top level and remove current element also from the top.

There are few operations perform by stack -

(i) push()

Insert the element in the stack.

(ii) pop()

remove the element in the stack.

(iii) peek()

This method is used to peek / see the topmost element in the stack.

(iv) isempty()

This method checks the stack is empty for deletion.

(v) Isfull()

This method checks the stack is full or not for insertion of other element.

Working of stack

- First we initialize pointer with -1 which track the top element in the stack.
- On pushing new element in stack we increment pointer top by 1 and place at new position.
- After reducing or delete the element from stack we reduce top by 1.
- Before pushing the element in the stack we check the stack is full or not.
- Before popping we check the stack is empty or not.

Ques. Write a program in C to implement stack using array.

```
#include <stdio.h>
#include <stdlib.h>
int stack[20], top = -1, n, i, num;
void push();
void pop();
void peek();
void display();
main()
{
    int ch;
    printf("Enter the total numbers in stack:");
    scanf("%d", &n);
    do
    {
        printf("Enter your choice:");
        scanf("%d", &ch);
        switch (ch)
        {
            case 1: push();
                    break;
```

```

case 2: pop();
        break;
case 3: peek();
        break;
case 4: display();
        break;
case 5: Exit(0);
        break;
default:
    printf("Invalid choice");
}
}
while (ch != 5);
return 0;
}
void push()
{
    if (top == n-1)
    {
        printf("Stack is full");
    }
    else
    {

```

```

        top++;
        printf("Enter the number:");
        scanf ("%d", &num);
        stack[top] = num;
    }
}
void pop()
{
    if (top == -1)
    {
        printf("stack is empty");
    }
    else
    {
        num = top;
        top--;
        printf("%d is pop", num);
    }
}
void peek();
{
    printf("%d", stack[top]);
}

```



```
void display()
```

```
{
    for(i=top; i>=0; i--)
    {
        printf("%d", stack[i]);
    }
}
```

Ques. What is stack data structure? Discuss the working principle of stack.

Ques. What is stack? Write a program in C to implement all the methods in stack.

Stack Application

Expression is a collection of operators and operand that represent a specific value.

Types of expression:-

- (i) Infix expression
- (ii) Postfix expression
- (iii) Prefix expression

Infix Expression

Those expression in which operators exist between operand is called infix expression.

Ex:- $A+B*C$

Postfix Expression

Those expression where operator reside after from the operand called postfix expression.

Ex:- $ABC*+$

Prefix Expression

Those expression where operators reside previous from the operand called prefix expression.

Ex:- $*+ABC$

Conversion of Infix to Postfix

Rule 1: Read all the symbols from left to right in the given expression.

Rule 2: If the reading symbol is operand then we directly print it to the result.

Rule 3: If the reading symbol is "(" and then push on to the stack.

Rule 4: If the reading symbol is ")" then pop all the contents of stack until respective "(" is popped out and print each pop symbol in the result.

Rule 5: If the reading symbol is operator then push it on to the stack. First pop the operator which is already on the stack that have high or equal precedence than current operator and print them in the result.

Ques. Write the given infix expression change to postfix expression.

(i) $a * (b + c) / d$

Symbol	Stack	Expression
a	—	a
*	*	a
(	*(	a
b	*(	ab
+	*(+	ab
c	*(+	abc
)	*	abc+
/	*/	abc+
d	*/	abc+d
Postfix expression —		abc+d/*

(ii) $A + (B * C - (D / E \wedge F) * G) * H$

Symbol	Stack	Expression
A	—	A
+	+	A
(	+(	A
B	+(	AB
*	+(*	AB
C	+(*	ABC

Symbol	Stack	Expression
-	+(-	ABC*
(	+(-(	ABC*
D	+(-(	ABC*D
/	+(-(	ABC*D
E	+(-(	ABC*DE
^	+(-(	ABC*DE
F	+(-(	ABC*DEF
)	+(-	ABC*DEF^/
*	+(-*	ABC*DEF^/
G	+(-*	ABC*DEF^/G
)	+	ABC*DEF^/G*-
*	+	ABC*DEF^/G*-
H	+	ABC*DEF^/G*-H

Postfix expression! - ABC*DEF^/G*-H*+

Ques. Convert the $((A+B)*(C+D)/(E-F))+G$ into postfix expression.

Symbol	Stack	Expression
(	(	
(	((	
A	((	A
+	((+	A
B	((+	AB
)	(	AB+
*	(*	AB+
(	(*(	AB+
C	(*(	AB+C
+	(*(+	AB+C
D	(*(+	AB+CD
)	(*	AB+CD+
/	(*/	AB+CD+
(	(*/(	AB+CD+
E	(*/(	AB+CD+E
-	(*/(-	AB+CD+E
F	(*/(-	AB+CD+EF
)	(*/	AB+CD+EF-
)		AB+CD+EF-/*
+	+	AB+CD+EF-/*

Symbol	Stack	Expression
G	+	AB+CD+EF-/*G
Postfix	Expression:-	AB+CD+EF-/*G+

Evaluation of Postfix Expression

Rule 1: We read the postfix expression from left to right.

Rule 2: If reading symbol is operand then push it onto stack.

Rule 3: If the reading symbol is operator then pop two successive operand, operand 1 and operand 2 from the stack and perform operation operand 2, operator, operand 1 and perform the operation and again push the resultant into the stack.

Rule 4: Finally, perform a pop operation and display the popped value as final result.

Ques. $(5+4) * (3-2)$

Symbol	Stack	Expression
(	(	5
5	(	5
+	(+)	5
4	(+)	54
)	—	54+
*	*	54+
(	*(	54+
3	*(	54+3
-	*(-	54+3
2	*(-	54+32
)	*	54+32-
conversion —		54+32-*

Evaluation —

Symbol	Stack	Postfix Evaluation
5	5	—
4	54	—
+	9	+
3	93	—
2	932	—
-	91	-
*	9	*

Ques. $5 + 4 * 3$

Ques. Evaluate the following postfix expression using stack.

8, 2, /, 3, *, 4, -, 6, 2, /, +

Ques. $4 + 3 * 3 / 4$

Ques. Find the value of given expression using Stack.

$(4 + 3) * 8 + 6 / 2$

Ques. Write a program in C to convert infix to postfix expression.

```
#include <stdio.h>
int stack[100], top = -1;
void push(char);
char pop();
int priority(char);
main()
```

```
{
    char exp[20], x, *e;
    printf("Enter the infix expression:");
    scanf("%s", &exp);
    e = exp;
    while(*e != '\0')
    {
        if(isalnum(*e))
        {
            printf("%c", *e);
        }
        else if(*e == '(')
        {
            push(*e);
        }
    }
}
```

```

else if (*e == '(')
{
    while ((x = pop()) != '(')
    {
        printf("%c", x);
    }
}
else
{
    while (top != -1 && priority(stack[top])
           >= priority(*e))
    {
        printf("%c", pop());
    }
    push(*e);
}
e++;
}
while (top != -1)
{
    printf("%c", pop());
}
return 0;

```

```

}
void push(char c)
{
    stack[++top] = c;
}
char pop()
{
    if (top == -1)
    {
        return -1;
    }
    else
    {
        return stack[top--];
    }
}
int int priority(char x)
{
    if (x == '(')
    {
        return 0;
    }
    else if (x == '+' || x == '-')

```



```

    {
        return 1;
    }
    else if (x == '*' || x == '/')
    {
        return 2;
    }
    else
    {
        return 3;
    }
    return 0;
}

```

Ques. write a program to evaluate any postfix expression in c language.

```

#include <stdio.h>
int stack[50], top = -1;
void push(int);
int pop();
main()

```

```

{
    char exp[20], *e;
    int n1, n2, n3;
    printf("Enter the postfix expression: ");
    scanf("%s", &exp);
    e = exp;
    while(*e != '\0')
    {
        if(isdigit(*e))
        {
            n3 = *e - 48;
            push(n3);
        }
        else
        {
            n1 = pop();

```

switch(*e)

5

Case '+': $n_3 = n_1 + n_2$,

```
break;
```

Case 1-1: $n_3 = n_1 - n_2$,

break;

Case ' * ' : $n_3 = n_1 * n_2$,

break;

case ' / ' : $n_3 = n_1/n_2$;

break;

default:

```
printf("Entered operator is wrong"),
```

2

2

push (m3),

 $e + t;$

3

```
printf ("The evaluated value of postfix  
expression %s is %d", exp, pop());
```

```
return 0;
```

3

void push (int s)

9

stack[++top] = s

3

```
int pop()
```

5

```
return stack [top--];
```

3

Conversion of Infix to Prefix expression

Rule 1: we read any infix expression from right to left.

Rule 2: If reading symbol is operand, print this symbol in expression.

Rule 3: If reading symbol is ')' then push in to stack.

Rule 4: If symbol is '(' then print all the symbols from stack till reach ')'.
 Rule 5: If symbol is ')' then pop the stack.

Rule 5: If reading symbol is operator then compare this symbol with top element of stack.

reading symbol precedence is more than top element of stack then push this symbol in stack, otherwise pop the element from the stack and print in expression, After this push operator in stack.

Rule 6: Finally, after stack is empty reverse again the expression. This expression is prefix notation of infix expression.

Ques. $(a+b)*c$

Symbol	Stack	Expression
c	—	c
*	*	c
)	*	c
b	*	cb
+	*)	cb
a	*)	cb a
(	*	cb a +

Prefix exp.:- $*+abc$

Ques. Convert the given expression into prefix.
 $((A+B)+(C+D)/(E-F))+G$

Symbol	Stack	Expression
G	—	G
+	+	G
)	+	G
)	++)	G
F	++)	G F
-	+))-	G F
E	+))-	G F E
(	+))	G F E -
/	+) /	G F E -
)	+) /)	G F E -
D	+) /)	G F E - D
+	+) /) +	G F E - D
C	+) /) +	G F E - D C
(	+) /	G F E - D C +
+	+) / *	G F E - D C +
)	+) / *)	G F E - D C +
B	+) / *)	G F E - D C + B
+	+) / *) +	G F E - D C + B
A	+) / *) +	G F E - D C + B

Queue

Symbol	Stack	Expression
(	+)/*	GFE-DC+BA+
(	+	GFE-DC+BA*/
		GFE-DC+BA*/+

Prefix Expression - +/*AB+CD-EFG

Queue is another special type of data structure where items are inserted at one end called rear and deleted at the other end called front. It works on the principle of FIFO (First In First Out).

There are two types of operation in queue:

- (i) enqueue (ii) dequeue

Where enqueue is a process where we insert data in queue and dequeue is another process where deleted element from the queue.

There are four types of queue -

- Linear queue
- Circular queue
- Priority queue
- Dequeue (Double-ended)

Utilisation of queue

- It is used to schedule the jobs to be processed by the CPU.

- Queue is used to search any element in graph data structure.
- When multiple users send print to printer, each printing job is kept in the printing queue.

Implementation of Linear queue using ~~stack~~ ^{array}.

```
#include <stdio.h>
#include <stdlib.h>
#define max 5
int lqueue [50];
void insert();
void delete();
void display();
int rear = -1, front = -1;
main()
{
    int ch;
    while (1)
    {
        printf("Enter your choice: ");
        scanf("%d", &ch);
        switch (ch)
        {
            case 1: insert();
                    break;
            case 2: delete();
                    break;
```

```

Case 3: display();
        break;
Case 4: exit(0);
    }
}
return 0;
}
void insert()
{
    int n;
    if (rear == max - 1)
    {
        printf("Queue is overflow");
    }
    printf("Enter the data: ");
    scanf("%d", &n);
    rear++;
    lqueue[rear] = n;
    if (front == -1)
    {
        front = 0;
    }
}
}

```

```

void delete()
{
    int n;
    if (front == -1)
    {
        printf("Queue is underflow");
    }
    n = lqueue[front];
    front++;
    printf("deleted element is %d:", n);
}
void display() {
    int n;
    if (front == -1) {
        printf("Queue is empty");
    }
    for (int i = front; i <= rear; i++)
    {
        printf("%d", lqueue[i]);
    }
    printf("\n");
}
}

```


Circular Queue

A circular queue is a data structure where elements are added from the front and remove from rear and which when full the last element of the circular queue is linked to the element preceding in the sequence. This structure resembles a circle and it also called the ring buffer.

Operations in circular queue

(A) Enqueue operation

In this process we insert elements in circular queue with the help of following steps-

(i) Check the queue is full or not

In this process we check $\text{size} == \text{capacity}$, that display the queue is full where insertion is not possible, overflow condition arise.

(ii) Initial condition

when the queue is empty, use the front

to track the first element of the queue and rear to track the last element of the queue. Initially we set both the pointers with -1 numerical value.

(iii) Insert new element

To add a new element in circular queue we increment the rear pointer with $\text{rear} = (\text{rear} + 1) \% \text{size}$.

(B) Dequeue operation

It is the process where we delete element from circular queue.

(i) Check the queue is empty or not, if front value is -1 then it indicates underflow condition.

(ii) we remove the element in circular queue with the help of front pointers where $\text{front} = (\text{front} + 1) \% \text{size}$.

(iii) If the element in a queue is last then is only one then we set the pointer

front and rear with numerical value -1.

Implementation of circular queue with arrays.

```
#include <stdio.h>
```

```
#define size 5
```

```
int queue[size], front = -1, rear = -1;
```

```
void enqueue()
```

```
{
```

```
    int n;
```

```
    if ((rear + 1) % size == front)
```

```
    {
```

```
        printf("queue is full");
```

```
    }
```

```
    if (front == -1) {
```

```
        front = 0;
```

```
    }
```

```
    rear = (rear + 1) % size;
```

```
    printf("enter the data: ");
```

```
    scanf("%d", &n);
```

```
    queue[rear] = n;
```

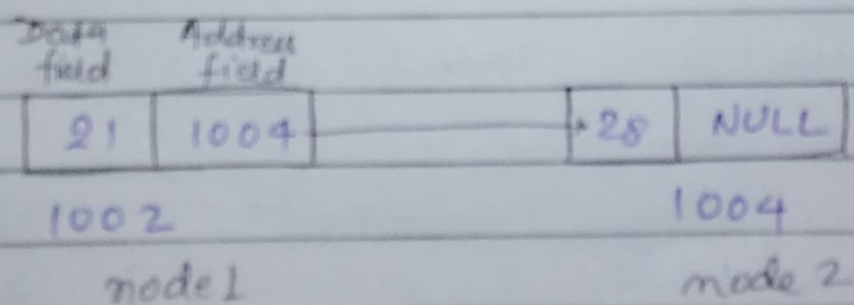
```
}
```

void dequeue()

```
{
```


LINKED LIST

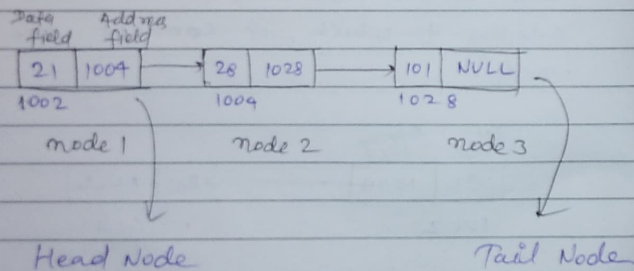
- A linked list is a linear data structure which consist of nodes which are connected each other with the help of links (pointers).
- These are not stored in continuous memory location. Each node consist of two fields where first field contain the data item and second field contain the memory address to which it connect the next node.



- In linked list data structure every memory is created during the running of program where we allocate or deallocate at runtime.
- We use linked list in various data

Structure like stack, queue, graph and hashmap, etc.

- Every first node in linked list is called head node which points to first node and last node of linked list called the tail node; which contain NULL value in its address field.



Types of Linked List

- (i) Single Linked List
- (ii) Doubly Linked List
- (iii) Circular linked list

Creation of nodes in linked list

struct node

```
{  
    int data;  
    struct node *next;  
};
```

Creation of node instance

struct node *head, *tail;

Initialisation of head and tail

head = NULL;

tail = NULL;

Insertion of elements or data in linked list

void insertfirst()

```
{  
    int value;  
    struct node *newnode;  
    newnode = (struct node node) malloc (sizeof (struct node));  
    if (newnode == NULL)  
    {  
        printf("Memory is full");  
    }
```



```

return;
}
printf("Enter the data:");
scanf("%d", &value);
newnode → data = value;
newnode → next = NULL;
if (head == NULL)
{
    head → next = NULL tail = newnode;
    tail → next = newnode;
}
else
{
    newnode → next = head;
    head = newnode;
}
printf("%d Data is insert\n", value);
}

```

void insertatlast()

```

{
    int value;
    struct node *temp, *newnode;

```

```

temp = head;
newnode = (struct node) malloc (size of (struct node));
if (newnode == NULL)
{
    printf("Memory is full");
    return;
}
printf("Enter the data:");
scanf("%d", &value);
newnode → data = value;
newnode → next = NULL;
if (head == NULL)
{
    head → next = newnode;
    tail → next = newnode;
}
else
{
    while (temp → next != NULL)
    {
        temp = temp → next;
    }
    temp → next = newnode;
}
printf("Inserted the element at last\n");
}

```

Void display()

```

{
    struct node * temp;
    if(head == NULL)
    {
        temp = head;
        while(temp != NULL)
        {
            temp = temp -> next;
            printf("%d", temp -> data);
        }
        printf("\n");
    }
}

```

Insert ^{element} at specified position in linked list

Void insertpos()

```

{
    int i, value, pos;
    struct node * temp; * newnode;
    newnode = malloc(sizeof(struct node));
    if(newnode == NULL)
    {
        printf("Memory is full");
        return;
    }
    printf("Enter the value:");
    scanf("%d", &value);
    newnode -> data = value;
    newnode -> next = NULL;
    printf("Enter the position:");
    scanf("%d", &pos);
    if(head == NULL)
    {
        head = newnode;
        tail = newnode;
    }
}

```


else

```

{
    temp = head;
    for (i = 2; i < pos; i++)
    {
        temp = temp → next;
    }
    newnode → next = temp → next;
    temp → next = newnode;
}
}

```

Void delete()

```

{
    struct node * temp;
    temp = head;
    while (temp → next != NULL) if (head == NULL)
    {
        printf("Underflow");
        return;
    }
    head = head → next;
    temp → next = NULL;
    free(temp);
}
}

```

Void deletepos()

```

{
    struct node * temp; * temp2;
    if (head == NULL)
    {
        temp = head;
        int i, pos;
        printf("Enter the position: ");
        scanf("%d", & pos);
        for (i = 2; i < pos; i++)
        {
            temp = temp → next;
        }
        temp2 = temp → next;
        temp → next = temp2 → next;
        free(temp2);
    }
}

```

Ques. WAP in C to implement single linked list having calling method -

- (i) Insert at start()
- (ii) Insert at last()
- (iii) Insert at specified()
- (iv) deletepos()
- (v) delete()
- (vi) display()